

Module 24: Copy Semantics

Introduction to C++ Programming — Part 3: Object-Oriented Programming with C++

Module Overview

This module resolves Module 23's cliffhanger: an RAI class that owns heap memory breaks the moment it is copied. Students learn the four everyday places copies happen, what the compiler's default memberwise copy does, and why copying an owning pointer creates a shallow copy that leads to shared data and double deletes. They then write a correct deep-copying copy constructor and copy assignment operator, meet the Rule of Three, see copy-and-swap, and learn to forbid copying with = delete. The module ends with its most important lesson: the Rule of Zero - design classes from members that manage themselves, and write none of this.

Suggested Duration: 90 minutes

Learning Objectives

By the end of this module, students will be able to:

- Identify the four situations in which an object is copied, and distinguish copy construction from copy assignment.
- Explain memberwise copy, and why it produces a shallow copy for owning raw pointers.
- Write a deep-copying copy constructor and a copy assignment operator that handles self-assignment and frees old data safely.
- State and apply the Rule of Three, and recognise the copy-and-swap idiom.
- Make a class non-copyable with = delete, and explain when that is the right design.
- Apply the Rule of Zero by using self-managing members such as `std::vector`.

Prerequisites

- Module 23 (destructors and RAI) - this module directly continues its `SampleBuffer` cliffhanger.
- Part 2 Module 10 (`new[]/delete[]`) and Module 13 (operator overloading syntax).

Materials & Instructor Prep

- 24-copy-semantics/examples/01_when_copies_happen.cpp - predict which special member function runs on each numbered line before running it.
- examples/02_shallow_copy_problem.cpp and 03_deep_copy_constructor.cpp for a back-to-back address comparison.
- Optional: build the exercise with `-fsanitize=address` to show a double-free report.

Key Vocabulary

Term	Definition
Copy constructor	A constructor taking a const reference to the same class, used to build a NEW object as a copy of an existing one.
Copy assignment operator	operator= taking a const reference to the same class, used to replace the contents of an object that ALREADY exists.
Memberwise copy	The compiler-generated copy: each member is copied using its own copy operation.
Shallow copy	A copy that duplicates a pointer's address but not the data it points to, so both objects share one resource.
Deep copy	A copy that allocates new storage and copies the pointed-to data, so the copies are independent.
Rule of Three	If a class needs a user-written destructor, copy constructor or copy assignment operator, it almost certainly needs all three.
Rule of Zero	Design classes whose members manage themselves, so none of the special member functions need to be written.

Lesson Outline & Instructor Talking Points

1. When do copies happen? (15 min)

- Run examples/01_when_copies_happen.cpp after students predict each numbered step. Most will miss the copy made by pass-by-value and by push_back.
- Draw the distinction on the board: Gradebook b = a; creates b (copy constructor); b = a; changes an existing b (copy assignment).

2. The shallow copy problem (15 min)

- Run examples/02_shallow_copy_problem.cpp. Point at the identical addresses, then at a.get(0) changing when only b was modified.
- Draw two boxes pointing at one array. Ask: what happens when both destructors run? Explain the double delete in words - the example deliberately has no destructor so it doesn't crash.

Instructor note: The diagram of two objects pointing at one array is the image students should leave with. Draw it large and leave it on the board for the rest of the session.

3. Writing the copy constructor and copy assignment (25 min)

- Live-code the copy constructor from examples/03. Ask why the parameter must be a reference (by value would need a copy, which needs the copy constructor...).
- Live-code copy assignment from examples/04 in four steps: self-assignment check, copy first, free second, return *this. Ask what goes wrong if the delete[] comes first and new then fails.

- Show copy-and-swap briefly as an elegant alternative; accept either in student work.

4. The Rule of Three and = delete (15 min)

- State the Rule of Three and treat any destructor that deletes something as an alarm bell.
- Run examples/05_non_copyable.cpp. Ask what it would mean to copy a SupplyGuard - and why a compile error is better than a hidden runtime bug.

5. The Rule of Zero (10 min)

- Run examples/06_rule_of_zero.cpp: a std::vector member, no special functions written, and copying is correct.
- Close with the guideline: in new code, prefer the Rule of Zero; reach for std::vector before new[].

Instructor note: *Don't let students leave thinking 'always write the Rule of Three'. The professional answer is usually to need none of it.*

Common Student Misconceptions

Misconception	How to Address It
"Gradebook b = a; uses the assignment operator."	It creates a new object, so it calls the copy constructor. Copy assignment only runs when the left-hand object already exists.
"The compiler's default copy is always wrong for classes with pointers."	It's wrong only when the pointer OWNS what it points to. A non-owning pointer (a borrow) is copied correctly as an address.
"In operator=, delete the old data first, then copy."	If the allocation then fails, the object is left with a dangling pointer. Copy first, then free the old data.
"Every class needs a copy constructor."	Classes built from std::string, std::vector and other self-managing members are copied correctly by default - the Rule of Zero.

Exercises & Module Project

Exercise: 24-copy-semantics/exercises/exercise1.cpp - add a deep-copying copy constructor and a safe copy assignment operator to Module 23's ReadingBuffer, then uncomment the independence, assignment and self-assignment tests; explain how the Rule of Zero would avoid the work. Project (three tracks): Generic - ScoreSheet 'what-if' moderation; EE - Waveform clipping experiments; Biomedical - ECG trace annotation on a copy of the medical record. Each does the Rule of Three by hand, makes the Module 23 RAll class non-copyable, then rewrites the data class with the Rule of Zero.

Facilitation guidance: Check the order inside every copy assignment (self-check, copy, free, assign, return *this), and run each submission with -fsanitize=address: a missing or wrong copy operation shows up immediately as a double-free. Reward the Rule-of-Zero comparison in Part 3 of the project - it's the lesson that matters most in practice.

Discussion & Comprehension Check Questions

1. Why must a copy constructor take its parameter by reference?

Guidance: Passing by value would itself require a copy, which would call the copy constructor, which would need another copy - infinitely. The compiler refuses to compile it.

2. What two bugs does a shallow copy of an owning pointer cause?

Guidance: Changes through one object appear in the other, and both destructors delete the same memory (a double delete), leaving a dangling pointer in between.

3. Why is `std::vector<double>` a better member than `double*` for most classes?

Guidance: `std::vector` already deep-copies, assigns and frees itself correctly, so the class needs no special member functions - the Rule of Zero.

Assessment Ideas

- Predict-the-call: a list of ten statements; students label each as copy constructor, copy assignment, or neither.
- Bug hunt: give a copy assignment operator that deletes before copying and has no self-check; students explain both bugs and fix them.

Answer Key Notes

See [24-copy-semantics/exercises/solutions/exercise1_solution.cpp](#).

Wrap-Up & Bridge to Next Module

Bridge forward: "Copies are now correct - but copying a million values out of an object that's about to be thrown away is pure waste. Next module shows how to move instead of copy."

Instructor Tips

- If time is short, protect the shallow-copy diagram (step 2) and the Rule of Zero (step 5).
- Run student code with AddressSanitizer during the session; seeing 'double-free' on their own code is memorable.